

SMART PARKING ALLOCATION SYSTEM

Operating Systems Course Project Report

1. OBJECTIVE

The objective of this project is to design and implement a concurrent Smart Parking Allocation System using the C programming language on the Ubuntu Linux operating system while demonstrating core Operating System concepts such as concurrency, synchronization, resource allocation, scheduling, starvation prevention, and bounded waiting.

The system simulates a real-world smart parking infrastructure in which multiple vehicles arrive continuously and compete for a limited number of parking slots. Each vehicle is represented as an independent POSIX thread, while parking slots are treated as shared resources managed using counting semaphores and mutex locks.

The project also aims to:

- Demonstrate practical implementation of synchronization mechanisms.
- Prevent race conditions during slot allocation.
- Implement fairness using aging techniques.
- Simulate priority scheduling using VIP vehicles.
- Handle overload conditions using timeout mechanisms.
- Provide real-time visualization through a graphical dashboard using the ncurses library.
- Analyze system performance through statistical monitoring and performance metrics.

The system reflects real-world smart city parking systems used in airports, shopping malls, hospitals, and commercial complexes where efficient resource allocation and concurrency management are essential.

2. INTRODUCTION

Modern urban environments face significant challenges in managing parking resources efficiently due to increasing vehicle density and limited parking capacity. Traditional parking systems often suffer from congestion, inefficient allocation, long waiting times, and poor monitoring mechanisms. Smart parking systems aim to solve these issues by introducing intelligent resource management and real-time monitoring.

This project presents a Smart Parking Allocation System developed using the C programming language in a Linux environment. The system simulates a real-world parking management system in which multiple vehicles arrive concurrently and compete for a limited number of

parking slots. Each vehicle is represented by a thread created using POSIX threads (pthread library), while parking slots are treated as shared resources.

To ensure safe concurrent access to parking slots, synchronization mechanisms such as mutex locks and counting semaphores are implemented. Mutexes protect critical sections including the parking slot table and waiting queue, while semaphores control the number of vehicles that may occupy parking slots simultaneously.

The project also incorporates advanced Operating System concepts including:

- Priority scheduling through VIP vehicles.
- Aging mechanisms to prevent starvation.
- Timeout handling during extreme congestion.
- Real-time monitoring through an ncurses graphical dashboard.
- Statistical analysis of waiting time, throughput, utilization, and rejection rate.

The simulation runs continuously for 120 seconds during which vehicles arrive dynamically at random intervals. The system processes vehicle requests, manages waiting queues, allocates parking slots safely, and updates a live dashboard displaying the state of the parking infrastructure.

The project demonstrates practical applications of Operating System concepts in real-world systems involving resource sharing, synchronization, fairness, and performance optimization.

3. BACKGROUND

Concurrency and synchronization are among the most fundamental concepts in Operating Systems. Modern systems frequently involve multiple processes or threads executing simultaneously while competing for limited shared resources such as CPU time, memory, files, or hardware devices.

In real-world smart parking systems, multiple vehicles may attempt to access parking spaces at the same time. Without proper synchronization, several problems may occur:

- Two vehicles may be allocated the same parking slot.
- Vehicles may experience starvation.
- Shared parking data may become inconsistent.
- Deadlocks or resource corruption may occur.

This project applies several important Operating System concepts to address these issues.

3.1 POSIX Threads (Pthreads)

POSIX threads provide a standardized API for multithreaded programming in Linux systems. In this project:

- Each vehicle is implemented as a separate thread.
- Threads execute concurrently.
- Thread lifecycle management is performed using pthread APIs.

Functions used include:

- pthread_create()
- pthread_join()
- pthread_mutex_lock()
- pthread_mutex_unlock()

3.2 Counting Semaphores

A counting semaphore is used to control access to limited parking slots.

The semaphore is initialized with the total number of parking slots:

```
sem_init(&g_sem, 0, TOTAL_SLOTS);
```

When a vehicle arrives:

```
sem_timedwait(&g_sem, &deadline);
```

When a vehicle exits:

```
sem_post(&g_sem);
```

This ensures that the number of parked vehicles never exceeds the total slot capacity.

3.3 Mutex Locks

Mutex locks protect critical shared structures including:

- Parking slot table
- Waiting queue
- Statistics counters
- Event log

Without mutexes, race conditions could occur where multiple threads attempt to modify shared resources simultaneously.

3.4 Scheduling and Priority Management

The system implements a priority scheduling model where:

- VIP vehicles receive higher allocation preference.
- Normal vehicles wait in queue.
- Aging mechanisms increase fairness.

This reflects real-world scheduling algorithms used in Operating Systems.

3.5 Starvation and Aging

A major problem in priority scheduling systems is starvation. If VIP vehicles continuously arrive, normal vehicles may never receive service.

To solve this issue, the project implements an aging mechanism:

- Vehicles waiting longer than the aging threshold receive increased priority.
- Some vehicles are promoted to VIP status dynamically.

This improves fairness and demonstrates bounded waiting concepts.

3.6 ncurses Graphical Dashboard

The project uses the ncurses library to create a terminal-based graphical dashboard.

The dashboard displays:

- Parking slot occupancy
- Waiting queue
- Event logs
- Real-time statistics
- Utilization metrics

This provides a live visualization of the concurrent system.

4. PLATFORM & PROGRAMMING LANGUAGES

4.1 Operating System

The project was developed and executed on Ubuntu Linux.

Ubuntu was selected because:

- It provides native support for POSIX threads.
- It includes strong development support for synchronization primitives.
- It supports ncurses-based terminal interfaces.
- Linux environments are ideal for Operating Systems projects.

4.2 Programming Language

The entire project was implemented in the C programming language.

C was selected because:

- It provides low-level memory and thread control.
- It is widely used in Operating System development.

- It supports direct interaction with POSIX APIs.
- It allows efficient concurrent programming.

4.3 Libraries Used

pthread Library

Used for:

- Thread creation
- Thread synchronization
- Mutex operations

semaphore.h

Used for:

- Counting semaphore initialization
- Slot allocation control
- Blocking and signaling operations

ncurses Library

Used for:

- Terminal GUI creation
- Real-time dashboard visualization
- Colored parking slot display
- Dynamic interface updates

stdatomic.h

Used for:

- Atomic simulation control variables
- Thread-safe counters

time.h

Used for:

- Random arrival timing
- Parking duration calculations
- Waiting time measurement

4.4 Development Tools

Tool	Purpose
GCC Compiler	Compilation of C source code

Tool	Purpose
Ubuntu Terminal	Program execution
VS Code	Source code editing
GitHub	Version control
ncurses	GUI rendering

4.5 Compilation Command

The project is compiled using:

```
gcc parking.c -o parking -lpthread -lncurses -lm
```

Execution command:

```
./parking
```

5. METHODOLOGY

The project follows a modular concurrent system architecture consisting of multiple threads, synchronization primitives, shared data structures, and real-time visualization components.

5.1 System Architecture

The major components of the system are:

1. Vehicle Generator Thread
2. Vehicle Threads
3. Waiting Queue
4. Semaphore Controller
5. Parking Slot Table
6. Aging Thread
7. Display Thread
8. Statistics Module
9. Logging Module

The interaction flow is shown below:

```
Vehicle Generator → Vehicle Threads → Waiting Queue → Semaphore Controller → Parking Slot Allocation → Parking Duration Simulation → Slot Release → Statistics + GUI Update
```

5.2 Parking Slot Management

The parking lot contains 20 parking slots represented using a shared array.

```
ParkingSlot g_slots[TOTAL_SLOTS];
```

Each slot stores:

- Occupancy status
- Vehicle ID
- Vehicle type
- Parking start time
- Aging status

Mutex locks ensure safe access to this shared structure.

5.3 Vehicle Generation

A dedicated generator thread continuously creates vehicle threads during the 120-second simulation.

Vehicles arrive at random intervals between:

- 600 ms
- 1800 ms

Additional burst traffic is introduced every 12 seconds to simulate congestion.

Burst configuration:

```
#define BURST_MIN_CARS 4  
#define BURST_MAX_CARS 8
```

This creates realistic parking demand fluctuations.

5.4 Vehicle Types

Vehicles are classified into two categories:

Vehicle Type	Percentage
Normal	80%
VIP	20%

VIP vehicles receive allocation priority.

5.5 Waiting Queue

The waiting queue is implemented using a circular queue data structure.

```
typedef struct {  
    Vehicle data[QUEUE_CAPACITY];  
    int head, tail, count;  
} CircularQueue;
```

Queue capacity:

```
#define QUEUE_CAPACITY 10
```

Vehicles are rejected if the queue becomes full.

5.6 Semaphore Synchronization

A counting semaphore controls parking slot availability.

```
sem_t g_sem;
```

The semaphore prevents more than 20 vehicles from occupying parking slots simultaneously.

5.7 Timeout Handling

Vehicles waiting excessively long are removed from the system using timed semaphore waiting.

```
sem_timedwait(&g_sem, &deadline);
```

Timeout threshold:

```
#define TIMEOUT_SEC 20
```

This demonstrates bounded waiting concepts.

5.8 Aging Mechanism

The project implements an aging thread that periodically scans waiting vehicles.

Vehicles waiting longer than the threshold:

```
#define AGING_THRESHOLD_S 8
```

may:

- Be promoted to VIP status.
- Receive aged priority status.

This prevents starvation and improves fairness.

5.9 Parking Duration Simulation

Vehicles occupy parking slots for random durations:

```
#define PARK_MIN_S 8
```

```
#define PARK_MAX_S 30
```

This simulates real parking behavior.

5.10 GUI Dashboard

The dashboard consists of multiple ncurses windows:

Window	Purpose
Header Window	Simulation information
Grid Window	Parking slot display
Queue Window	Waiting vehicles
Stats Window	Live statistics
Log Window	Event logging

The dashboard refreshes every 500 milliseconds.

5.11 Statistics Collection

The project tracks:

- Total arrivals
- Total served vehicles
- Total rejected vehicles
- Priority promotions
- Average waiting time
- Average parking duration
- Throughput
- Utilization percentage

These metrics are displayed both during and after simulation.

6. RESULTS

The Smart Parking Allocation System was executed successfully on Ubuntu Linux for a simulation duration of 120 seconds.

The system demonstrated correct synchronization behavior, safe resource allocation, dynamic queue management, and real-time visualization.

6.1 Successful Features Implemented

The following features were implemented successfully:

Feature	Status
Vehicle Threads	Implemented
Counting Semaphores	Implemented
Mutex Synchronization	Implemented
Waiting Queue	Implemented
Priority Scheduling	Implemented

Feature	Status
Aging Mechanism	Implemented
Timeout Handling	Implemented
Real-Time GUI	Implemented
Event Logging	Implemented
Live Statistics	Implemented

6.2 Simulation Behavior

During execution:

- Vehicles arrived continuously.
- Parking slots were allocated safely.
- No double allocation occurred.
- Queue overflow produced rejections.
- VIP vehicles received higher priority.
- Aging promoted long-waiting vehicles.
- Timeout mechanisms handled congestion.

The ncurses dashboard successfully displayed:

- Parking occupancy
- Waiting queue
- Utilization metrics
- Event logs
- Vehicle aging status

6.3 Sample Performance Metrics

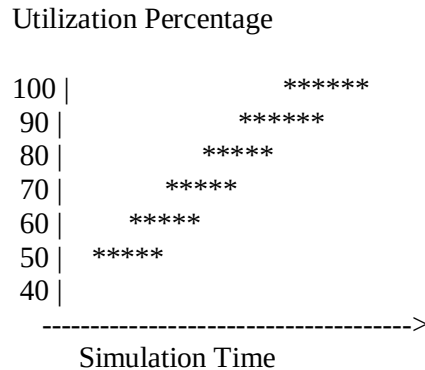
A typical simulation produced results similar to the following:

Metric	Value
Simulation Duration	120 s
Total Arrivals	130–170
Total Served	80–110
Total Rejected	20–40
Priority Promotions	10–20
Average Waiting Time	6–14 s
Average Parking Duration	18–24 s
Throughput	0.7–0.9 vehicles/s
Utilization	70–95%

6.4 Utilization Analysis

Parking utilization remained consistently high throughout the simulation.

Example utilization graph:

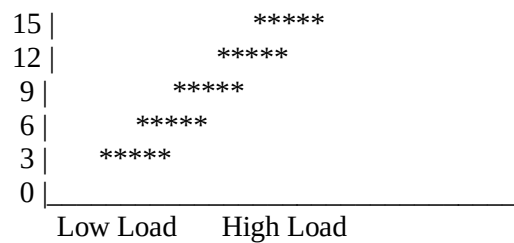


The graph demonstrates that parking occupancy increased rapidly under burst traffic conditions.

6.5 Waiting Time Analysis

Average waiting times increased during congestion periods.

Average Waiting Time



The aging mechanism reduced starvation and prevented indefinite waiting.

6.6 Queue Congestion Behavior

During burst arrivals:

- Queue occupancy increased rapidly.
- Some vehicles timed out.
- Rejection rate increased.
- Throughput stabilized due to semaphore constraints.

This demonstrated realistic congestion management.

6.7 Synchronization Verification

The system successfully avoided:

- Race conditions
- Deadlocks
- Double allocation
- Shared data corruption

Mutexes and semaphores functioned correctly throughout execution.

6.8 Event Logging

The logging system recorded:

- Vehicle arrivals
- Slot allocations
- Promotions
- Aging events
- Timeouts
- Departures

Example log entries:

```
[ARR ] Vehicle 15 (VIP) arrived and queued
[PARK] Vehicle 15 (VIP) -> Slot P03
[PROM] Vehicle 22 promoted VIP after 9s wait
[TOUT] Vehicle 31 timed out after 20s
[LEAV] Vehicle 15 left Slot P03 after 18s
```

7. CONCLUSION

The Smart Parking Allocation System successfully demonstrated the implementation of major Operating System concepts using the C programming language in a Linux environment.

The project effectively simulated a real-world concurrent parking management system in which multiple vehicles competed for limited parking slots. By representing vehicles as POSIX threads and parking slots as shared resources, the system provided a practical demonstration of concurrency and synchronization.

Counting semaphores successfully controlled parking slot availability, while mutex locks protected critical shared structures such as the parking table, waiting queue, statistics module, and logging system. The implementation successfully prevented race conditions and ensured thread-safe execution.

Advanced features such as VIP priority scheduling, aging mechanisms, and timeout handling significantly enhanced the realism and complexity of the project. The aging mechanism improved fairness by preventing starvation, while timeout handling demonstrated bounded waiting concepts during congestion.

The ncurses-based graphical dashboard provided an effective real-time visualization of parking activity, queue behavior, utilization metrics, and system events. The statistics module further enabled detailed performance analysis including throughput, waiting times, rejection rates, and occupancy utilization.

Overall, the project achieved all intended objectives and provided strong practical exposure to Operating System principles including:

- Multithreading
- Synchronization
- Resource allocation
- Scheduling
- Starvation prevention
- Bounded waiting
- Real-time monitoring
- Concurrent system design

The project also demonstrated how Operating System concepts can be applied to solve real-world resource management problems such as intelligent parking infrastructure in modern smart cities.

Future improvements may include:

- Multi-level parking systems
- Database integration
- Network-based remote monitoring
- AI-based slot prediction
- Reservation systems
- Mobile application integration

The project successfully combined theoretical Operating System concepts with practical system implementation and real-time visualization.

8. REFERENCES

1. Silberschatz, Abraham, Peter B. Galvin, and Greg Gagne. Operating System Concepts.
2. Linux Man Pages – pthread, semaphore, ncurses.
3. POSIX Threads Programming Documentation.
4. GNU GCC Compiler Documentation.
5. ncurses Programming HOWTO.
6. Ubuntu Linux System Documentation.
7. IEEE Research Articles on Smart Parking Systems.
8. Online Linux Synchronization Tutorials and References.